Compiler Design Lab – Project Review 1 Report

Project Title:

DAG-Based Optimization for Basic Blocks

Team Members:

- Abhinav Gupta (24BCT0185)
- Shobit Khanna (24BCT0187)
- Arnav Goel (24BCT0189)

1. Problem Statement

Unoptimized intermediate code, such as standard Three-Address Code (TAC), often contains redundant computations and common subexpressions. Values are computed multiple times, wasting CPU cycles and memory bandwidth. Our goal is to build an optimizer that constructs a Directed Acyclic Graph (DAG) for basic blocks to automatically detect and eliminate these redundant patterns.

Challenges:

- Efficiently detecting duplicate computations without traversing the entire graph.
- Preventing the creation of duplicate leaf nodes for reused variables.
- Generating valid, shorter TAC from the finalized graph structure.

Naive Input vs Optimized Output:

```
t1 = a + b
t2 = c + d
t3 = a + b
```

```
t1 = a + b
t2 = c + d
t3 = t1
```

2. Objectives

Review 1 Objectives:

- Establish core data structures for DAG nodes.
- Implement Value Numbering using hash maps for constant-time O(1) duplicate detection.
- Demonstrate successful graph construction and redundancy elimination on hardcoded basic blocks.

Later Reviews:

- Develop a file parser to read raw TAC text files dynamically.
- Implement a graph traversal function to generate and output the final optimized TAC.
- Conduct edge-case testing and deliver the final report.

3. Literature Survey

Theme	Books/Papers	Key Idea	Relevance to this Project
Foundations	A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman (2007). *Compilers: Principles, techniques, & tools*.	Constructing DAGs for basic blocks to expose common subexpressions.	Serves as the core theoretical foundation for our graph architecture.
Implementation	D. Watson (2017). *A Practical Approach to Compiler Construction*.	Mapping intermediate representations to efficient data structures.	Guides our translation of TAC string instructions into C++ node objects.
Algorithm Speed	S. Muchnick (1997). *Advanced Compiler Design and Implementation*.	Using "Value Numbering" (hash signatures) to identify expressions locally.	Provides the basis for our $O(1)$ hash map lookup logic.

4. System Design

Pipeline Architecture:

Input TAC text file → Token Parser → DAG Constructor (Value Numbering) → Code Generator → Output Optimized TAC file.

Core Optimizations Targeted:

1. Common Subexpression Elimination.
2. Dead Code Elimination (Future Scope).
3. Variable Tracking and Pointer Reassignment.

5. Local Optimizations Breakdown

Optimization	What it does	Before & After

Common Subexpression Elimination	Drops the recalculation of an expression if the exact operands and operator were already processed.	`[Before]`
		`t1 = a + b`
		`t3 = a + b`
		
		`[After]`
		`t1 = a + b`
		`t3 = t1`

Redundant Leaf Prevention	Reuses existing variable nodes rather than allocating new memory for duplicate operands.	`[Before]`
		`t1 = a + b`
		`t2 = a - c`
		
		`[After]`
		`Both t1 and t2 map to the single leaf node 'a'.`

6. Core Mechanism: Value Numbering

Rather than looping through previous instructions to find a match (which scales poorly), our optimizer relies on a **Signature Hash Map** strategy:

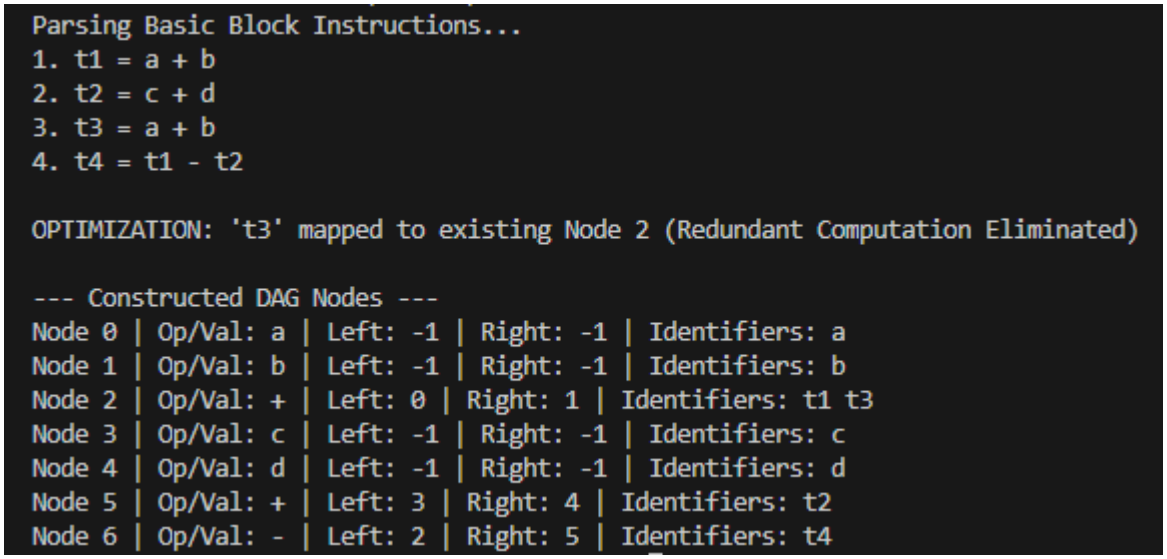
- **Window:** The compiler processes one TAC instruction at a time.
- **Signature Generation:** When processing t1 = a + b, the program identifies the node IDs for a (e.g., Node 0) and b (e.g., Node 1). It creates a unique string signature representing this operation: +,0,1.

- **Constant-Time Check:** It checks the central hash map for the key +,0,1. If it already exists, the computation is fully redundant and is skipped. If not, a new node is created in the DAG and the signature is saved.

7. Implementation Status

- **Data Structures:** The DAGNode struct and critical hash maps (valueNumbering, varToNode) are fully implemented.
- **Constructor Logic:** The core logic to generate signatures and correctly link parent/child nodes is actively working.
- **Environment:** Written cleanly in C++ using standard libraries.
- **Current State (25%):** The driver successfully processes hardcoded TAC instructions and proves duplicate detection by mapping redundant variables to existing Node IDs without generating duplicate graphs.

Output:



8. Remaining Work and Conclusion

Remaining Work:

- **File I/O:** Replace the hardcoded instruction strings with a robust text file parser.
- **Code Generation:** Write the traversal algorithm to walk the finalized DAG and print the new optimized Three-Address Code.
- **Testing:** Measure instruction counts before and after optimization to empirically prove efficiency gains.

Conclusion:

The foundational graph architecture and the O(1) Value Numbering search algorithm have been successfully implemented. The system accurately reads intermediate code and links redundant variables to existing memory nodes rather than duplicating them. The next phase will focus on generating the final output text and hardening the parser against edge cases.